SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
SIMATS ENGINEERING
COMPUTER SCIENCE AND ENGINEERING

LABORATORY RECORD

SUB CODE & NAME: CSA17 - ARTIFICIAL INTELLIGENCE

Name : **Richard Benson Raj.B.A**

Register Number : **1925260077**

Course : **Artificial Intelligence**

Course Code : **CSA1711**

Dept : **BTECH Artificial Intelligence Machine Learning**

INDEX

S.No	List of Programs	Page	Signature
1	8-Puzzle Problem	3	
2	8-Queen Problem	5	
3	Water Jug Problem	7	
4	Crypt-Arithmetic Problem	9	
5	Missionaries and Cannibals Problem	11	
6	Vacuum Cleaner Problem	13	
7	Breadth First Search (BFS)	15	
8	Depth First Search (DFS)	16	
9	Travelling Salesman Problem	17	
10	A* Algorithm	19	
11	Map Coloring (Constraint Satisfaction Problem)	21	
12	Tic Tac Toe Game	23	
13	Minimax Algorithm	26	
14	Alpha-Beta Pruning	27	
15	Decision Tree	29	
16	Feed Forward Neural Network	31	
17	Sum of the Integers from 1 to n	33	
18	Database with Name and DOB	34	
19	STUDENT-TEACHER-SUB-CODE Database	36	
20	PLANETS Database	38	
21	Towers of Hanoi	40	
22	Bird can Fly or Not	41	
23	Family Tree	43	
24	Dieting System based on Disease	45	
25	Monkey Banana Problem	47	
26	Fruit and its Colour using Backtracking	49	
27	Best First Search Algorithm	51	
28	Medical Diagnosis	53	
29	Forward Chaining	55	
30	Backward Chaining	57	
31	Web Blog using WordPress (Anchor Tag, Title Tag, etc.)	59	
32	Pattern Matching	61	
33	Number of Vowels	63	

EXPERIMENT 1 8-Puzzle Problem

1. AIM

To write a Python program to solve the 8-Puzzle problem using the A* search algorithm with the Manhattan distance heuristic.

2. PROCEDURE

1. Represent each board configuration as a tuple of 9 numbers (0 denotes the blank tile).
2. Define the goal state (1 2 3 / 4 5 6 / 7 8 _) and the Manhattan distance heuristic h(n).
3. Generate successor states by sliding the blank tile Up, Down, Left or Right.
4. Keep a priority queue ordered by f(n) = g(n) + h(n) and always expand the state with the least f value.
5. Stop when the goal state is removed from the queue and print the sequence of moves and boards.

3. CODE (Python)

```
import heapq

GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def manhattan(state):
    dist = 0
    for i, v in enumerate(state):
        if v != 0:
            g = v - 1
            dist += abs(i // 3 - g // 3) + abs(i % 3 - g % 3)
    return dist

def neighbours(state):
    z = state.index(0)
    r, c = divmod(z, 3)
    for dr, dc, name in ((-1, 0, "Up"), (1, 0, "Down"), (0, -1, "Left"), (0, 1, "Right")):
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            s = list(state)
            k = nr * 3 + nc
            s[z], s[k] = s[k], s[z]
            yield tuple(s), name

def solve(start):
    pq = [(manhattan(start), 0, start, [])]
    best = {start: 0}
    while pq:
        f, g, state, path = heapq.heappop(pq)
        if state == GOAL:
            return path
        for nxt, move in neighbours(state):
            if nxt not in best or g + 1 < best[nxt]:
                best[nxt] = g + 1
                heapq.heappush(pq, (g + 1 + manhattan(nxt), g + 1, nxt, path + [(move, nxt)]))
    return None

def show(state):
    for i in range(0, 9, 3):
        print(" ".join(str(x) if x else "_" for x in state[i:i + 3]))

start = (1, 2, 3, 5, 0, 6, 4, 7, 8)
print("Initial state:")
show(start)
solution = solve(start)
for step, (move, state) in enumerate(solution, 1):
    print("\nStep", step, ": move blank", move)
    show(state)
print("\nPuzzle solved in", len(solution), "moves")
```

4. INPUT

```
Initial state = (1, 2, 3, 5, _, 6, 4, 7, 8)   Goal state = (1, 2, 3, 4, 5, 6, 7, 8, _)
```

5. OUTPUT

```
Initial state:
1 2 3
5 _ 6
4 7 8

Step 1 : move blank Left
1 2 3
_ 5 6
4 7 8

Step 2 : move blank Down
1 2 3
4 5 6
_ 7 8

Step 3 : move blank Right
1 2 3
4 5 6
7 _ 8

Step 4 : move blank Right
1 2 3
4 5 6
7 8 _

Puzzle solved in 4 moves
```

6. RESULT

The 8-Puzzle problem was solved successfully using A* search and the optimal sequence of moves was displayed.

EXPERIMENT 2 8-Queen Problem

1. AIM

To write a Python program to place 8 queens on an 8x8 chessboard so that no two queens attack each other, using backtracking.

2. PROCEDURE

1. Place queens row by row; store the column of the queen of each row in a list.
2. For a new queen, check that no earlier queen is in the same column or on the same diagonal.
3. If a safe column exists, place the queen and move to the next row.
4. If no column is safe, backtrack: remove the previous queen and try its next column.
5. When all 8 rows are filled, a solution is found; count all solutions and print the first board.

3. CODE (Python)

```
N = 8

def safe(cols, row, col):
    for r, c in enumerate(cols):
        if c == col or abs(c - col) == abs(r - row):
            return False
    return True

def place(cols, solutions):
    row = len(cols)
    if row == N:
        solutions.append(cols[:])
        return
    for col in range(N):
        if safe(cols, row, col):
            cols.append(col)
            place(cols, solutions)
            cols.pop()

solutions = []
place([], solutions)
print("Total number of solutions for", N, "queens :", len(solutions))
first = solutions[0]
print("First solution (column of queen in each row):", first)
for r in range(N):
    print(" ".join("Q" if first[r] == c else "." for c in range(N)))
```

4. INPUT

```
N = 8 (size of the chessboard / number of queens)
```

5. OUTPUT

```
Total number of solutions for 8 queens : 92
First solution (column of queen in each row): [0, 4, 7, 5, 2, 6, 1, 3]
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
```

6. RESULT

All 92 solutions of the 8-Queens problem were found using backtracking and the first solution was displayed.

EXPERIMENT 3 Water Jug Problem

1. AIM

To write a Python program to measure exactly 2 litres of water using a 4-litre jug and a 3-litre jug, using Breadth First Search.

2. PROCEDURE

1. Represent a state as (x, y), the water in the 4-litre and 3-litre jugs; the start state is (0, 0).
2. Define the operations: fill a jug, empty a jug, pour one jug into the other.
3. Use a queue (BFS) to explore the states and store the parent of each visited state.
4. Stop when either jug contains the target quantity.
5. Trace back through the parents to print the sequence of states from the start state.

3. CODE (Python)

```
from collections import deque

def water_jug(cap_a, cap_b, target):
    start = (0, 0)
    parent = {start: None}
    queue = deque([start])
    while queue:
        a, b = queue.popleft()
        if a == target or b == target:
            path, s = [], (a, b)
            while s is not None:
                path.append(s)
                s = parent[s]
            return path[::-1]
        pour_ab = min(a, cap_b - b)
        pour_ba = min(b, cap_a - a)
        moves = [(cap_a, b), (a, cap_b), (0, b), (a, 0),
                 (a - pour_ab, b + pour_ab), (a + pour_ba, b - pour_ba)]
        for s in moves:
            if s not in parent:
                parent[s] = (a, b)
                queue.append(s)
    return None

cap_a, cap_b, target = 4, 3, 2
path = water_jug(cap_a, cap_b, target)
print("Jug capacities:", cap_a, "L and", cap_b, "L   Target:", target, "L")
for step, (a, b) in enumerate(path):
    print("Step", step, ": 4L jug =", a, " 3L jug =", b)
print("Target obtained in", len(path) - 1, "operations")
```

4. INPUT

```
Jug A capacity = 4 L, Jug B capacity = 3 L, Target = 2 L
```

5. OUTPUT

```
Jug capacities: 4 L and 3 L   Target: 2 L
Step 0 : 4L jug = 0  3L jug = 0
Step 1 : 4L jug = 0  3L jug = 3
Step 2 : 4L jug = 3  3L jug = 0
Step 3 : 4L jug = 3  3L jug = 3
Step 4 : 4L jug = 4  3L jug = 2
Target obtained in 4 operations
```

6. RESULT

Exactly 2 litres of water was measured using the 4-litre and 3-litre jugs in the minimum number of steps.

EXPERIMENT 4 Crypt-Arithmetic Problem

1. AIM

To write a Python program to solve the crypt-arithmetic puzzle SEND + MORE = MONEY.

2. PROCEDURE

1. Collect the distinct letters of the puzzle (there are 8 letters).
2. Try every assignment of distinct digits 0-9 to the letters using permutations.
3. Reject assignments in which the leading letter of any word is 0.
4. Convert each word to a number and check whether SEND + MORE = MONEY.
5. Print the digit assigned to each letter and the numeric addition.

3. CODE (Python)

```
from itertools import permutations

def solve(w1, w2, result):
    letters = sorted(set(w1 + w2 + result))
    leading = {w1[0], w2[0], result[0]}
    for perm in permutations(range(10), len(letters)):
        m = dict(zip(letters, perm))
        if any(m[ch] == 0 for ch in leading):
            continue
        value = lambda w: int("".join(str(m[ch]) for ch in w))
        if value(w1) + value(w2) == value(result):
            return m, value(w1), value(w2), value(result)
    return None

word1, word2, total = "SEND", "MORE", "MONEY"
sol = solve(word1, word2, total)
if sol:
    m, a, b, c = sol
    print("Letter -> digit mapping:")
    for ch in sorted(m):
        print(" ", ch, "=", m[ch])
    print(word1, "+", word2, "=", total)
    print(a, "+", b, "=", c)
else:
    print("No solution exists")
```

4. INPUT

```
word1 = SEND, word2 = MORE, result = MONEY
```

5. OUTPUT

```
Letter -> digit mapping:
  D = 7
  E = 5
  M = 1
  N = 6
  O = 0
  R = 8
  S = 9
  Y = 2
SEND + MORE = MONEY
9567 + 1085 = 10652
```

6. RESULT

The crypt-arithmetic puzzle was solved and the unique digit assignment (9567 + 1085 = 10652) was obtained.

EXPERIMENT 5 Missionaries and Cannibals Problem

1. AIM

To write a Python program to move 3 missionaries and 3 cannibals across a river using a boat of capacity 2 such that cannibals never outnumber missionaries on any bank.

2. PROCEDURE

1. Represent a state as (M, C, B): missionaries and cannibals on the left bank and the boat side (1 = left, 0 = right).
2. The start state is (3, 3, 1) and the goal state is (0, 0, 0).
3. Allowed boat loads are (1,0), (2,0), (0,1), (0,2) and (1,1).
4. A state is valid only if missionaries are not outnumbered on either bank (unless there are no missionaries there).
5. Use Breadth First Search to find the shortest sequence of crossings and print it.

3. CODE (Python)

```
from collections import deque

LOADS = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]

def valid(m, c):
    if not (0 <= m <= 3 and 0 <= c <= 3):
        return False
    if m != 0 and c > m:
        return False
    if 3 - m != 0 and 3 - c > 3 - m:
        return False
    return True

def bfs():
    start, goal = (3, 3, 1), (0, 0, 0)
    parent = {start: None}
    queue = deque([start])
    while queue:
        state = queue.popleft()
        if state == goal:
            path = []
            while state:
                path.append(state)
                state = parent[state]
            return path[::-1]
        m, c, b = state
        for dm, dc in LOADS:
            nm, nc = (m - dm, c - dc) if b == 1 else (m + dm, c + dc)
            nxt = (nm, nc, 1 - b)
            if valid(nm, nc) and nxt not in parent:
                parent[nxt] = state
                queue.append(nxt)

path = bfs()
print("Left bank (M, C)   Boat    Right bank (M, C)")
for m, c, b in path:
    print("   (%d, %d)          %-6s     (%d, %d)" % (m, c, "Left" if b else "Right", 3 - m, 3 - c))
print("Solved in", len(path) - 1, "crossings")
```

4. INPUT

```
Number of missionaries = 3, cannibals = 3, boat capacity = 2
```

5. OUTPUT

```
Left bank (M, C)   Boat    Right bank (M, C)
   (3, 3)          Left       (0, 0)
   (3, 1)          Right      (0, 2)
   (3, 2)          Left       (0, 1)
   (3, 0)          Right      (0, 3)
   (3, 1)          Left       (0, 2)
   (1, 1)          Right      (2, 2)
   (2, 2)          Left       (1, 1)
   (0, 2)          Right      (3, 1)
   (0, 3)          Left       (3, 0)
   (0, 1)          Right      (3, 2)
   (1, 1)          Left       (2, 2)
   (0, 0)          Right      (3, 3)
Solved in 11 crossings
```

6. RESULT

All missionaries and cannibals were moved safely to the right bank in the minimum number of crossings.

EXPERIMENT 6 Vacuum Cleaner Problem

1. AIM

To write a Python program to simulate a simple reflex vacuum cleaner agent in a two-room environment (rooms A and B).

2. PROCEDURE

1. Represent the environment by the status (Clean / Dirty) of rooms A and B and the agent's location.
2. If the current room is dirty, the agent performs the SUCK action.
3. If the current room is clean and the other room is dirty, the agent MOVEs to the other room.
4. If both rooms are clean, the agent STOPs.
5. Run the agent for different initial conditions and print every action and the total actions taken.

3. CODE (Python)

```
def vacuum_agent(location, status):
    rooms = dict(status)
    other = {"A": "B", "B": "A"}
    actions = 0
    print("Initial state: location =", location, ", rooms =", rooms)
    while True:
        if rooms[location] == "Dirty":
            rooms[location] = "Clean"
            actions += 1
            print("  SUCK  in room", location, "->", rooms)
        elif all(v == "Clean" for v in rooms.values()):
            print("  STOP  - both rooms are clean")
            break
        else:
            location = other[location]
            actions += 1
            print("  MOVE  to room", location)
    print("Total actions performed:", actions)

print("Test case 1")
vacuum_agent("A", {"A": "Dirty", "B": "Dirty"})
print("\nTest case 2")
vacuum_agent("B", {"A": "Dirty", "B": "Clean"})
```

4. INPUT

```
Case 1: location = A, A = Dirty, B = Dirty.   Case 2: location = B, A = Dirty, B = Clean
```

5. OUTPUT

```
Test case 1
Initial state: location = A , rooms = {'A': 'Dirty', 'B': 'Dirty'}
  SUCK  in room A -> {'A': 'Clean', 'B': 'Dirty'}
  MOVE  to room B
  SUCK  in room B -> {'A': 'Clean', 'B': 'Clean'}
  STOP  - both rooms are clean
Total actions performed: 3

Test case 2
Initial state: location = B , rooms = {'A': 'Dirty', 'B': 'Clean'}
  MOVE  to room A
  SUCK  in room A -> {'A': 'Clean', 'B': 'Clean'}
  STOP  - both rooms are clean
Total actions performed: 2
```

6. RESULT

The reflex vacuum cleaner agent cleaned both rooms in each test case and stopped when the environment was clean.

EXPERIMENT 7 Breadth First Search (BFS)

1. AIM

To write a Python program to implement the Breadth First Search (BFS) graph traversal algorithm.

2. PROCEDURE

1. Represent the graph as an adjacency list (dictionary).
2. Insert the start node into a queue and mark it as visited.
3. Remove the front node of the queue, print it and add all its unvisited neighbours to the queue.
4. Repeat until the queue becomes empty.
5. The order of printing is the BFS traversal order (level by level).

3. CODE (Python)

```
from collections import deque

graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [],
    "E": ["F"],
    "F": [],
}

def bfs(graph, start):
    visited = [start]
    queue = deque([start])
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for nb in graph[node]:
            if nb not in visited:
                visited.append(nb)
                queue.append(nb)
    return order

print("Graph (adjacency list):", graph)
print("BFS traversal starting from A:", " -> ".join(bfs(graph, "A")))
```

4. INPUT

```
Graph: A-{B,C}, B-{D,E}, C-{F}, E-{F};  Start node = A
```

5. OUTPUT

```
Graph (adjacency list): {'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F'], 'D': [], 'E': ['F'], 'F': []}
BFS traversal starting from A: A -> B -> C -> D -> E -> F
```

6. RESULT

The BFS traversal of the graph was obtained successfully.

EXPERIMENT 8 Depth First Search (DFS)

1. AIM

To write a Python program to implement the Depth First Search (DFS) graph traversal algorithm.

2. PROCEDURE

1. Represent the graph as an adjacency list (dictionary).
2. Start from the source node, mark it visited and print it.
3. Recursively visit each unvisited neighbour of the current node.
4. When a node has no unvisited neighbours, backtrack to the previous node.
5. The order of printing is the DFS traversal order.

3. CODE (Python)

```
graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [],
    "E": ["F"],
    "F": [],
}

def dfs(graph, node, visited, order):
    visited.add(node)
    order.append(node)
    for nb in graph[node]:
        if nb not in visited:
            dfs(graph, nb, visited, order)
    return order

print("Graph (adjacency list):", graph)
print("DFS traversal starting from A:", " -> ".join(dfs(graph, "A", set(), [])))
```

4. INPUT

```
Graph: A-{B,C}, B-{D,E}, C-{F}, E-{F};  Start node = A
```

5. OUTPUT

```
Graph (adjacency list): {'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F'], 'D': [], 'E': ['F'], 'F': []}
DFS traversal starting from A: A -> B -> D -> E -> F -> C
```

6. RESULT

The DFS traversal of the graph was obtained successfully.

EXPERIMENT 9 Travelling Salesman Problem

1. AIM

To write a Python program to find the shortest route that visits every city exactly once and returns to the starting city (Travelling Salesman Problem).

2. PROCEDURE

1. Represent the cities and the distances between them as a distance matrix.
2. Fix city 0 as the starting city and generate all permutations of the remaining cities.
3. For every permutation compute the total cost of the round trip.
4. Keep track of the route with the minimum cost.
5. Print the optimal route and its cost.

3. CODE (Python)

```
from itertools import permutations

cities = ["A", "B", "C", "D"]
dist = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0],
]

def route_cost(route):
    cost = 0
    for i in range(len(route) - 1):
        cost += dist[route[i]][route[i + 1]]
    return cost

best_cost, best_route = float("inf"), None
for perm in permutations(range(1, len(cities))):
    route = (0,) + perm + (0,)
    cost = route_cost(route)
    print("Route", " -> ".join(cities[i] for i in route), " cost =", cost)
    if cost < best_cost:
        best_cost, best_route = cost, route

print("\nShortest route :", " -> ".join(cities[i] for i in best_route))
print("Minimum cost   :", best_cost)
```

4. INPUT

```
Cities A, B, C, D with distance matrix [[0,10,15,20],[10,0,35,25],[15,35,0,30],[20,25,30,0]]; start city = A
```

5. OUTPUT

```
Route A -> B -> C -> D -> A  cost = 95
Route A -> B -> D -> C -> A  cost = 80
Route A -> C -> B -> D -> A  cost = 95
Route A -> C -> D -> B -> A  cost = 80
Route A -> D -> B -> C -> A  cost = 95
Route A -> D -> C -> B -> A  cost = 95

Shortest route : A -> B -> D -> C -> A
Minimum cost   : 80
```

6. RESULT

The shortest tour of the cities was found using the brute-force method and its minimum cost was displayed.

EXPERIMENT 10

A* Algorithm

1. AIM

To write a Python program to implement the A* search algorithm to find the least-cost path in a weighted graph.

2. PROCEDURE

1. Represent the graph as a weighted adjacency dictionary and give a heuristic value h(n) for every node.
2. Insert the start node into a priority queue with f(n) = g(n) + h(n).
3. Remove the node with the smallest f value; if it is the goal, return the path and its cost.
4. Otherwise expand it, computing g and f for every neighbour and inserting them into the queue.
5. Repeat until the goal is reached or the queue becomes empty.

3. CODE (Python)

```
import heapq

graph = {
    "S": {"A": 1, "B": 4},
    "A": {"B": 2, "C": 5, "D": 12},
    "B": {"C": 2},
    "C": {"D": 3},
    "D": {},
}
h = {"S": 7, "A": 6, "B": 4, "C": 2, "D": 0}

def a_star(start, goal):
    open_list = [(h[start], 0, start, [start])]
    closed = {}
    while open_list:
        f, g, node, path = heapq.heappop(open_list)
        if node == goal:
            return path, g
        if node in closed and closed[node] <= g:
            continue
        closed[node] = g
        print("Expanding", node, ": g =", g, ", h =", h[node], ", f =", f)
        for nb, w in graph[node].items():
            heapq.heappush(open_list, (g + w + h[nb], g + w, nb, path + [nb]))
    return None, float("inf")

path, cost = a_star("S", "D")
print("\nOptimal path:", " -> ".join(path))
print("Path cost   :", cost)
```

4. INPUT

```
Weighted graph S-A(1), S-B(4), A-B(2), A-C(5), A-D(12), B-C(2), C-D(3); heuristic h = {S:7, A:6, B:4, C:2, D:0}; start = S, goal = D
```

5. OUTPUT

```
Expanding S : g = 0 , h = 7 , f = 7
Expanding A : g = 1 , h = 6 , f = 7
Expanding B : g = 3 , h = 4 , f = 7
Expanding C : g = 5 , h = 2 , f = 7

Optimal path: S -> A -> B -> C -> D
Path cost   : 8
```

6. RESULT

The A* algorithm found the least-cost path from the start node to the goal node.

EXPERIMENT 11

Map Coloring (Constraint Satisfaction Problem)

1. AIM

To write a Python program to colour the map of Australia using three colours such that no two adjacent regions have the same colour, by modelling it as a Constraint Satisfaction Problem (CSP).

2. PROCEDURE

1. Variables: the regions WA, NT, SA, Q, NSW, V, T. Domain: {Red, Green, Blue}.
2. Constraint: two neighbouring regions must not be assigned the same colour.
3. Select an unassigned region and try each colour in its domain.
4. Assign a colour only if it is consistent with the colours of the already coloured neighbours.
5. If no colour is possible, backtrack and change the previous assignment; stop when all regions are coloured.

3. CODE (Python)

```
neighbors = {
    "WA": ["NT", "SA"],
    "NT": ["WA", "SA", "Q"],
    "SA": ["WA", "NT", "Q", "NSW", "V"],
    "Q": ["NT", "SA", "NSW"],
    "NSW": ["Q", "SA", "V"],
    "V": ["SA", "NSW"],
    "T": [],
}
colors = ["Red", "Green", "Blue"]

def consistent(region, color, assignment):
    return all(assignment.get(n) != color for n in neighbors[region])

def backtrack(assignment):
    if len(assignment) == len(neighbors):
        return assignment
    region = next(r for r in neighbors if r not in assignment)
    for color in colors:
        if consistent(region, color, assignment):
            assignment[region] = color
            result = backtrack(assignment)
            if result:
                return result
            del assignment[region]
    return None

solution = backtrack({})
if solution:
    for region, color in solution.items():
        print(region, "->", color)
else:
    print("No solution")
```

4. INPUT

```
Regions: WA, NT, SA, Q, NSW, V, T with their adjacency list; Colours: Red, Green, Blue
```

5. OUTPUT

```
WA -> Red
NT -> Green
SA -> Blue
Q -> Red
NSW -> Green
V -> Red
T -> Red
```

6. RESULT

The map was coloured with three colours satisfying all adjacency constraints using a CSP with backtracking.

EXPERIMENT 12

Tic Tac Toe Game

1. AIM

To write a Python program to play the Tic Tac Toe game between a human player (X) and the computer (O).

2. PROCEDURE

1. Represent the 3x3 board as a list of 9 cells numbered 1 to 9.
2. Read the human move, check it is valid and place X on the board.
3. The computer chooses its move using the Minimax algorithm and places O.
4. After every move check the eight winning lines and whether the board is full.
5. Display the board after each move and announce the winner or a draw.

3. CODE (Python)

```
WIN = [(0, 1, 2), (3, 4, 5), (6, 7, 8), (0, 3, 6), (1, 4, 7), (2, 5, 8), (0, 4, 8), (2, 4, 6)]

def winner(b):
    for x, y, z in WIN:
        if b[x] != " " and b[x] == b[y] == b[z]:
            return b[x]
    return None

def minimax(b, comp_turn):
    w = winner(b)
    if w == "O":
        return 1
    if w == "X":
        return -1
    if " " not in b:
        return 0
    scores = []
    for i in range(9):
        if b[i] == " ":
            b[i] = "O" if comp_turn else "X"
            scores.append(minimax(b, not comp_turn))
            b[i] = " "
    return max(scores) if comp_turn else min(scores)

def best_move(b):
    best, move = -2, None
    for i in range(9):
        if b[i] == " ":
            b[i] = "O"
            score = minimax(b, False)
            b[i] = " "
            if score > best:
                best, move = score, i
    return move

def show(b):
    print(" %s | %s | %s " % tuple(b[0:3]))
    print("---+---+---")
    print(" %s | %s | %s " % tuple(b[3:6]))
    print("---+---+---")
    print(" %s | %s | %s " % tuple(b[6:9]))

board = [" "] * 9
print("You are X, computer is O. Cells are numbered 1-9 (left to right, top to bottom).\n")
while True:
    move = int(input("Your move (1-9): ")) - 1
    if move not in range(9) or board[move] != " ":
```

```
        print("Invalid move, try again.")
        continue
    print("You chose:", move + 1)
    board[move] = "X"
    if winner(board) or " " not in board:
        show(board)
        break
    comp = best_move(board)
    board[comp] = "O"
    print("Computer chose:", comp + 1)
    show(board)
    print()
    if winner(board) or " " not in board:
        break
w = winner(board)
print("\nResult:", ("Winner is " + w) if w else "The game is a draw")
```

4. INPUT

Human moves entered from the keyboard (see output below).

5. OUTPUT

```
You are X, computer is O. Cells are numbered 1-9 (left to right, top to bottom).

Your move (1-9): You chose: 1
Computer chose: 5
 X |   |
---+---+---
   | O |
---+---+---
   |   |

Your move (1-9): You chose: 2
Computer chose: 3
 X | X | O
---+---+---
   | O |
---+---+---
   |   |

Your move (1-9): You chose: 7
Computer chose: 4
 X | X | O
---+---+---
 O | O |
---+---+---
 X |   |

Your move (1-9): You chose: 6
Computer chose: 8
 X | X | O
---+---+---
 O | O | X
---+---+---
 X | O |

Your move (1-9): You chose: 9
 X | X | O
---+---+---
 O | O | X
---+---+---
 X | O | X

Result: The game is a draw
```

6. RESULT

The Tic Tac Toe game was implemented; the computer used the Minimax algorithm and the game result was announced.

EXPERIMENT 13

Minimax Algorithm

1. AIM

To write a Python program to implement the Minimax algorithm for two-player games and find the optimal value of a game tree.

2. PROCEDURE

1. Represent the game tree as a list of leaf (utility) values of a complete binary tree.
2. At a MAX node choose the maximum of the values of its children; at a MIN node choose the minimum.
3. Recursively evaluate the tree from the root (MAX) down to the leaves.
4. Return the leaf value when the maximum depth is reached.
5. Print the optimal value obtained at the root and the number of leaf nodes evaluated.

3. CODE (Python)

```
import math

leaf_count = 0

def minimax(depth, node, is_max, scores, height):
    global leaf_count
    if depth == height:
        leaf_count += 1
        return scores[node]
    left = minimax(depth + 1, node * 2, not is_max, scores, height)
    right = minimax(depth + 1, node * 2 + 1, not is_max, scores, height)
    return max(left, right) if is_max else min(left, right)

scores = [3, 5, 6, 9, 1, 2, 0, -1]
height = int(math.log2(len(scores)))
print("Leaf values :", scores)
print("Tree height :", height)
print("The optimal value (root = MAX) is:", minimax(0, 0, True, scores, height))
print("Leaf nodes evaluated:", leaf_count)
```

4. INPUT

```
Leaf values of the game tree = [3, 5, 6, 9, 1, 2, 0, -1]; root is a MAX node
```

5. OUTPUT

```
Leaf values : [3, 5, 6, 9, 1, 2, 0, -1]
Tree height : 3
The optimal value (root = MAX) is: 5
Leaf nodes evaluated: 8
```

6. RESULT

The optimal value of the game tree was computed using the Minimax algorithm.

EXPERIMENT 14

Alpha-Beta Pruning

1. AIM

To write a Python program to implement the Alpha-Beta pruning algorithm for game playing and compare it with Minimax.

2. PROCEDURE

1. Use the same game tree as in the Minimax program and initialise alpha = -infinity and beta = +infinity.
2. At a MAX node update alpha = max(alpha, value); at a MIN node update beta = min(beta, value).
3. If beta <= alpha, the remaining children of the node cannot affect the result, so prune them.
4. Count the leaf nodes evaluated to show the saving over plain Minimax.
5. Print the optimal value at the root.

3. CODE (Python)

```
import math

leaf_count = 0

def alphabeta(depth, node, is_max, scores, height, alpha, beta):
    global leaf_count
    if depth == height:
        leaf_count += 1
        return scores[node]
    if is_max:
        best = -math.inf
        for child in (node * 2, node * 2 + 1):
            val = alphabeta(depth + 1, child, False, scores, height, alpha, beta)
            best = max(best, val)
            alpha = max(alpha, best)
            if beta <= alpha:
                break
        return best
    else:
        best = math.inf
        for child in (node * 2, node * 2 + 1):
            val = alphabeta(depth + 1, child, True, scores, height, alpha, beta)
            best = min(best, val)
            beta = min(beta, best)
            if beta <= alpha:
                break
        return best

scores = [3, 5, 6, 9, 1, 2, 0, -1]
height = int(math.log2(len(scores)))
print("Leaf values :", scores)
print("The optimal value (Alpha-Beta) is:", alphabeta(0, 0, True, scores, height, -math.inf, math.inf))
print("Leaf nodes evaluated:", leaf_count, "out of", len(scores))
print("Leaf nodes pruned   :", len(scores) - leaf_count)
```

4. INPUT

```
Leaf values of the game tree = [3, 5, 6, 9, 1, 2, 0, -1]; alpha = -infinity, beta = +infinity
```

5. OUTPUT

```
Leaf values : [3, 5, 6, 9, 1, 2, 0, -1]
The optimal value (Alpha-Beta) is: 5
Leaf nodes evaluated: 5 out of 8
Leaf nodes pruned   : 3
```

6. RESULT

Alpha-Beta pruning gave the same optimal value as Minimax while evaluating fewer leaf nodes.

EXPERIMENT 15

Decision Tree

1. AIM

To write a Python program to build a Decision Tree classifier for the Play-Tennis data set and use it to predict new samples.

2. PROCEDURE

1. Create the training data (Outlook, Temperature, Humidity, Wind -> Play) and encode the categorical values as numbers.
2. Create a DecisionTreeClassifier using the entropy (information gain) criterion.
3. Train the model using the fit() method.
4. Display the learned tree rules using export_text().
5. Predict the class of new samples and display the training accuracy.

3. CODE (Python)

```
from sklearn.tree import DecisionTreeClassifier, export_text

outlook = {"Sunny": 0, "Overcast": 1, "Rain": 2}
temp = {"Hot": 0, "Mild": 1, "Cool": 2}
humidity = {"High": 0, "Normal": 1}
wind = {"Weak": 0, "Strong": 1}

data = [
    ("Sunny", "Hot", "High", "Weak", "No"),
    ("Sunny", "Hot", "High", "Strong", "No"),
    ("Overcast", "Hot", "High", "Weak", "Yes"),
    ("Rain", "Mild", "High", "Weak", "Yes"),
    ("Rain", "Cool", "Normal", "Weak", "Yes"),
    ("Rain", "Cool", "Normal", "Strong", "No"),
    ("Overcast", "Cool", "Normal", "Strong", "Yes"),
    ("Sunny", "Mild", "High", "Weak", "No"),
    ("Sunny", "Cool", "Normal", "Weak", "Yes"),
    ("Rain", "Mild", "Normal", "Weak", "Yes"),
    ("Sunny", "Mild", "Normal", "Strong", "Yes"),
    ("Overcast", "Mild", "High", "Strong", "Yes"),
    ("Overcast", "Hot", "Normal", "Weak", "Yes"),
    ("Rain", "Mild", "High", "Strong", "No"),
]

def encode(row):
    return [outlook[row[0]], temp[row[1]], humidity[row[2]], wind[row[3]]]

X = [encode(r) for r in data]
y = [r[4] for r in data]

model = DecisionTreeClassifier(criterion="entropy", random_state=0)
model.fit(X, y)

print("Decision tree rules:")
print(export_text(model, feature_names=["Outlook", "Temperature", "Humidity", "Wind"]))

tests = [("Sunny", "Cool", "High", "Strong"), ("Overcast", "Mild", "Normal", "Weak"),
         ("Rain", "Mild", "High", "Strong")]
for t in tests:
    print("Prediction for", t, "->", model.predict([encode(t)])[0])
print("Training accuracy:", model.score(X, y))
```

4. INPUT

14 training samples of the Play-Tennis data set; test samples: (Sunny, Cool, High, Strong), (Overcast, Mild, Normal, Weak), (Rain, Mild, High, Strong)

5. OUTPUT

```
Decision tree rules:
|--- Humidity <= 0.50
|   |--- Outlook <= 0.50
|   |   |--- class: No
|   |--- Outlook >  0.50
|   |   |--- Outlook <= 1.50
|   |   |   |--- class: Yes
|   |   |--- Outlook >  1.50
|   |   |   |--- Wind <= 0.50
|   |   |   |   |--- class: Yes
|   |   |   |--- Wind >  0.50
|   |   |   |   |--- class: No
|--- Humidity >  0.50
|   |--- Wind <= 0.50
|   |   |--- class: Yes
|   |--- Wind >  0.50
|   |   |--- Outlook <= 1.50
|   |   |   |--- class: Yes
|   |   |--- Outlook >  1.50
|   |   |   |--- class: No

Prediction for ('Sunny', 'Cool', 'High', 'Strong') -> No
Prediction for ('Overcast', 'Mild', 'Normal', 'Weak') -> Yes
Prediction for ('Rain', 'Mild', 'High', 'Strong') -> No
Training accuracy: 1.0
```

6. RESULT

A decision tree classifier was built and trained, and it predicted the class of the new samples correctly.

EXPERIMENT 16

Feed Forward Neural Network

1. AIM

To write a Python program to implement a Feed Forward Neural Network with back-propagation to learn the XOR function.

2. PROCEDURE

1. Create the XOR training data and initialise the weights and biases of a 2-4-1 network randomly.
2. Forward pass: compute the hidden layer and output layer activations using the sigmoid function.
3. Compute the error between the target and the output.
4. Backward pass: propagate the error back and update the weights using gradient descent.
5. Repeat for many epochs and print the loss and the final predictions.

3. CODE (Python)

```
import numpy as np

np.random.seed(1)
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
y = np.array([[0], [1], [1], [0]], dtype=float)

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

W1 = np.random.randn(2, 4)
b1 = np.zeros((1, 4))
W2 = np.random.randn(4, 1)
b2 = np.zeros((1, 1))
lr = 0.5

for epoch in range(1, 10001):
    h = sigmoid(X @ W1 + b1)
    out = sigmoid(h @ W2 + b2)
    error = y - out
    d_out = error * out * (1 - out)
    d_h = (d_out @ W2.T) * h * (1 - h)
    W2 += lr * h.T @ d_out
    b2 += lr * d_out.sum(axis=0, keepdims=True)
    W1 += lr * X.T @ d_h
    b1 += lr * d_h.sum(axis=0, keepdims=True)
    if epoch % 2000 == 0:
        print("Epoch", epoch, " Mean squared error =", round(float(np.mean(error ** 2)), 5))

print("\nFinal predictions:")
out = sigmoid(sigmoid(X @ W1 + b1) @ W2 + b2)
for x, o in zip(X, out):
    print(x.astype(int), "->", round(float(o[0]), 4), "-> class", int(o[0] > 0.5))
```

4. INPUT

```
XOR inputs [0,0], [0,1], [1,0], [1,1] with targets 0, 1, 1, 0; learning rate = 0.5; epochs = 10000
```

5. OUTPUT

```
Epoch 2000  Mean squared error = 0.0023
Epoch 4000  Mean squared error = 0.00081
Epoch 6000  Mean squared error = 0.00047
Epoch 8000  Mean squared error = 0.00033
Epoch 10000  Mean squared error = 0.00025

Final predictions:
[0 0] -> 0.0085 -> class 0
[0 1] -> 0.9855 -> class 1
[1 0] -> 0.9837 -> class 1
[1 1] -> 0.0213 -> class 0
```

6. RESULT

The feed forward neural network was trained with back-propagation and correctly learned the XOR function.

EXPERIMENT 17

Sum of the Integers from 1 to n

1. AIM

To write a Prolog program to find the sum of the integers from 1 to n using recursion.

2. PROCEDURE

1. Write the base case: the sum of the integers up to 0 is 0.
2. Write the recursive rule: sum(N, S) = sum(N-1) + N, for N > 0.
3. Open SWI-Prolog, save the program as a .pl file and load it with ?- [filename]. (consult).
4. Enter the queries with different values of n.
5. Observe the value returned in the output variable S.

3. CODE (Prolog)

```
% Sum of the integers from 1 to n
sum(0, 0) :- !.
sum(N, S) :-
    N > 0,
    N1 is N - 1,
    sum(N1, S1),
    S is S1 + N.
```

4. INPUT

```
?- sum(5, S).
?- sum(10, S).
?- sum(100, S).
?- sum(0, S).
```

5. OUTPUT

```
?- sum(5, S).
S = 15.

?- sum(10, S).
S = 55.

?- sum(100, S).
S = 5050.

?- sum(0, S).
S = 0.
```

6. RESULT

The sum of the integers from 1 to n was computed successfully using a recursive Prolog program.

EXPERIMENT 18

Database with Name and DOB

1. AIM

To write a Prolog program to create a database of persons with their names and dates of birth and to query it.

2. PROCEDURE

1. Represent each person as a fact person(Name, date(Day, Month, Year)).
2. Write the rule dob/2 to display the date of birth in the form D/M/Y.
3. Write the rule born_in_year/2 to find the persons born in a given year.
4. Open SWI-Prolog, save the program as a .pl file and load it with ?- [filename]. (consult).
5. Enter the queries and observe the answers.

3. CODE (Prolog)

```
% Facts : person(Name, date(Day, Month, Year))
person(arun,   date(15, 8, 2003)).
person(bala,   date(2, 3, 2002)).
person(chitra, date(21, 11, 2003)).
person(divya,  date(9, 1, 2004)).
person(eshwar, date(30, 6, 2002)).

% Rules
dob(Name, D/M/Y) :- person(Name, date(D, M, Y)).
born_in_year(Year, Name) :- person(Name, date(_, _, Year)).
```

4. INPUT

```
?- dob(arun, X).
?- dob(chitra, X).
?- dob(john, X).
?- findall(N, person(N, _), L).
?- findall(N, born_in_year(2003, N), L).
?- findall(N, born_in_year(2002, N), L).
```

5. OUTPUT

```
?- dob(arun, X).
X = 15/8/2003.

?- dob(chitra, X).
X = 21/11/2003.

?- dob(john, X).
false.

?- findall(N, person(N, _), L).
L = [arun, bala, chitra, divya, eshwar].

?- findall(N, born_in_year(2003, N), L).
L = [arun, chitra].

?- findall(N, born_in_year(2002, N), L).
L = [bala, eshwar].
```

6. RESULT

A database of names and dates of birth was created in Prolog and queried successfully.

EXPERIMENT 19

STUDENT-TEACHER-SUB-CODE Database

1. AIM

To write a Prolog program to create a database relating students, teachers and subject codes and to answer queries on it.

2. PROCEDURE

1. Create facts for teaches(Teacher, SubCode), subject(SubCode, SubjectName) and studies(Student, SubCode).
2. Write the rule student_teacher(S, T) to find the teacher of a student through the subject code.
3. Write the rule student_subject(S, Name) to find the subject names studied by a student.
4. Open SWI-Prolog, save the program as a .pl file and load it with ?- [filename]. (consult).
5. Enter the queries and observe the answers.

3. CODE (Prolog)

```
% teaches(Teacher, SubCode)
teaches(rajesh, csa17).
teaches(meena,  csa18).
teaches(kumar,  csa19).

% subject(SubCode, SubjectName)
subject(csa17, 'Artificial Intelligence').
subject(csa18, 'Machine Learning').
subject(csa19, 'Data Structures').

% studies(Student, SubCode)
studies(arun,   csa17).
studies(bala,   csa17).
studies(chitra, csa18).
studies(divya,  csa19).
studies(arun,   csa19).

% Rules
student_teacher(S, T) :- studies(S, C), teaches(T, C).
student_subject(S, Name) :- studies(S, C), subject(C, Name).
```

4. INPUT

```
?- subject(csa17, N).
?- findall(S, studies(S, csa17), L).
?- findall(T, student_teacher(arun, T), L).
?- findall(N, student_subject(arun, N), L).
?- findall(S-T, student_teacher(S, T), L).
```

5. OUTPUT

```
?- subject(csa17, N).
N = 'Artificial Intelligence'.

?- findall(S, studies(S, csa17), L).
L = [arun, bala].

?- findall(T, student_teacher(arun, T), L).
L = [rajesh, kumar].

?- findall(N, student_subject(arun, N), L).
L = ['Artificial Intelligence', 'Data Structures'].

?- findall(S-T, student_teacher(S, T), L).
L = [arun-rajesh, bala-rajesh, chitra-meena, divya-kumar, arun-kumar].
```

6. RESULT

The student-teacher-subject code database was created and the required queries were answered.

EXPERIMENT 20

PLANETS Database

1. AIM

To write a Prolog program to create a database of the planets of the solar system and to query their properties.

2. PROCEDURE

1. Create facts planet(Name, Position, Diameter_km, Type) for the eight planets.
2. Write rules larger_than/2, largest/1 and nth_planet/2 using the facts.
3. Open SWI-Prolog, save the program as a .pl file and load it with ?- [filename]. (consult).
4. Enter queries to retrieve the details of a planet, planets of a given type and comparisons.
5. Observe the answers.

3. CODE (Prolog)

```
% planet(Name, Position from Sun, Diameter in km, Type)
planet(mercury, 1,   4879, terrestrial).
planet(venus,   2,  12104, terrestrial).
planet(earth,   3,  12756, terrestrial).
planet(mars,    4,   6792, terrestrial).
planet(jupiter, 5, 142984, gas_giant).
planet(saturn,  6, 120536, gas_giant).
planet(uranus,  7,  51118, ice_giant).
planet(neptune, 8,  49528, ice_giant).

% Rules
larger_than(X, Y) :- planet(X, _, DX, _), planet(Y, _, DY, _), DX > DY.
largest(N) :- findall(D-P, planet(P, _, D, _), L), max_member(_-N, L).
nth_planet(Pos, Name) :- planet(Name, Pos, _, _).
```

4. INPUT

```
?- planet(earth, P, D, T).
?- findall(N, planet(N, _, _, gas_giant), L).
?- findall(N, planet(N, _, _, terrestrial), L).
?- larger_than(jupiter, earth).
?- larger_than(mars, earth).
?- largest(P).
?- findall(N, nth_planet(4, N), L).
```

5. OUTPUT

```
?- planet(earth, P, D, T).
P = 3,
D = 12756,
T = terrestrial.

?- findall(N, planet(N, _, _, gas_giant), L).
L = [jupiter, saturn].

?- findall(N, planet(N, _, _, terrestrial), L).
L = [mercury, venus, earth, mars].

?- larger_than(jupiter, earth).
true.

?- larger_than(mars, earth).
false.

?- largest(P).
P = jupiter.
```

```
?- findall(N, nth_planet(4, N), L).
L = [mars].
```

6. RESULT

The planets database was created and queried successfully.

EXPERIMENT 21

Towers of Hanoi

1. AIM

To write a Prolog program to solve the Towers of Hanoi problem for n disks.

2. PROCEDURE

1. Base case: with 0 disks there is nothing to move.
2. Recursive case: move N-1 disks from the source to the helper peg, move the largest disk to the destination, then move the N-1 disks from the helper to the destination.
3. Print each move using format/2.
4. Open SWI-Prolog, save the program as a .pl file and load it with ?- [filename]. (consult).
5. Query hanoi(N, From, To, Via) with different values of N.

3. CODE (Prolog)

```
% hanoi(N, From, To, Via)
hanoi(0, _, _, _) :- !.
hanoi(N, From, To, Via) :-
    N > 0,
    N1 is N - 1,
    hanoi(N1, From, Via, To),
    format('Move disk ~w from ~w to ~w~n', [N, From, To]),
    hanoi(N1, Via, To, From).
```

4. INPUT

```
?- hanoi(2, a, c, b).
?- hanoi(3, left, right, center).
```

5. OUTPUT

```
?- hanoi(2, a, c, b).
Move disk 1 from a to b
Move disk 2 from a to c
Move disk 1 from b to c
true.

?- hanoi(3, left, right, center).
Move disk 1 from left to right
Move disk 2 from left to center
Move disk 1 from right to center
Move disk 3 from left to right
Move disk 1 from center to left
Move disk 2 from center to right
Move disk 1 from left to right
true.
```

6. RESULT

The Towers of Hanoi problem was solved; n disks require 2^n - 1 moves (7 moves for 3 disks).

EXPERIMENT 22

Bird can Fly or Not

1. AIM

To write a Prolog program to print whether a particular bird can fly or not and to incorporate the required queries.

2. PROCEDURE

1. Create the facts bird(Name) for the known birds.
2. Create the facts cannot_fly(Name) for the birds that cannot fly (exceptions).
3. Write the rule can_fly(X): X is a bird and X is not an exception (negation as failure).
4. Open SWI-Prolog, save the program as a .pl file and load it with ?- [filename]. (consult).
5. Query print_fly(Bird) and the other queries and observe the messages.

3. CODE (Prolog)

```
bird(sparrow).
bird(eagle).
bird(penguin).
bird(ostrich).
bird(crow).
bird(parrot).

cannot_fly(penguin).
cannot_fly(ostrich).

can_fly(X) :- bird(X), \+ cannot_fly(X).

print_fly(X) :- can_fly(X), !, write(X), write(' can fly.'), nl.
print_fly(X) :- bird(X), cannot_fly(X), !, write(X), write(' cannot fly.'), nl.
print_fly(X) :- \+ bird(X), write(X), write(' is not a bird.'), nl.
```

4. INPUT

```
?- print_fly(sparrow).
?- print_fly(penguin).
?- print_fly(dog).
?- findall(B, can_fly(B), L).
?- findall(B, (bird(B), cannot_fly(B)), L).
```

5. OUTPUT

```
?- print_fly(sparrow).
sparrow can fly.
true.

?- print_fly(penguin).
penguin cannot fly.
true.

?- print_fly(dog).
dog is not a bird.
true.

?- findall(B, can_fly(B), L).
L = [sparrow, eagle, crow, parrot].

?- findall(B, (bird(B), cannot_fly(B)), L).
L = [penguin, ostrich].
```

6. RESULT

The program correctly printed whether a bird can fly or not, and the required queries were answered.

EXPERIMENT 23

Family Tree

1. AIM

To write a Prolog program to implement a family tree and to find relationships such as father, mother, grandparent, sibling and uncle.

2. PROCEDURE

1. Create facts male/1, female/1 and parent(Parent, Child) describing the family.
2. Define rules for father, mother, grandparent, sibling, brother and uncle in terms of the basic facts.
3. Open SWI-Prolog, save the program as a .pl file and load it with ?- [filename]. (consult).
4. Enter queries using findall/3 and setof/3 to list the relatives.
5. Observe the answers.

3. CODE (Prolog)

```
% Facts
male(john).   male(david).  male(peter).  male(jack).  male(tom).
female(mary). female(susan). female(anna). female(lisa).

parent(john, david).   parent(mary, david).
parent(john, susan).   parent(mary, susan).
parent(david, peter).  parent(anna, peter).
parent(david, lisa).   parent(anna, lisa).
parent(susan, tom).    parent(jack, tom).

% Rules
father(X, Y)      :- male(X), parent(X, Y).
mother(X, Y)      :- female(X), parent(X, Y).
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
sibling(X, Y)     :- parent(P, X), parent(P, Y), X \= Y.
brother(X, Y)     :- male(X), sibling(X, Y).
uncle(X, Y)       :- parent(P, Y), brother(X, P).
```

4. INPUT

```
?- findall(C, father(john, C), L).
?- findall(C, mother(mary, C), L).
?- findall(G, grandparent(G, peter), L).
?- setof(A, grandparent(john, A), L).
?- setof(S, sibling(david, S), L).
?- setof(U, uncle(U, tom), L).
```

5. OUTPUT

```
?- findall(C, father(john, C), L).
L = [david, susan].

?- findall(C, mother(mary, C), L).
L = [david, susan].

?- findall(G, grandparent(G, peter), L).
L = [john, mary].

?- setof(A, grandparent(john, A), L).
L = [lisa, peter, tom].

?- setof(S, sibling(david, S), L).
L = [susan].

?- setof(U, uncle(U, tom), L).
L = [david].
```

6. RESULT

The family tree was implemented and the relationships were derived using Prolog rules.

EXPERIMENT 24

Dieting System based on Disease

1. AIM

To write a Prolog program to suggest a suitable diet for a given disease.

2. PROCEDURE

1. Create facts diet(Disease, RecommendedFoods) and avoid(Disease, FoodsToAvoid).
2. Write the rule suggest(Disease) that prints the recommended diet and the foods to avoid.
3. Add a second clause to give a message when the disease is not in the knowledge base.
4. Open SWI-Prolog, save the program as a .pl file and load it with ?- [filename]. (consult).
5. Query suggest(Disease) for different diseases.

3. CODE (Prolog)

```
diet(diabetes,     'Low-sugar foods, whole grains, green vegetables and fibre').
diet(hypertension, 'Low-salt food, fruits, vegetables and low-fat dairy').
diet(anaemia,      'Iron-rich foods such as spinach, dates, beetroot and pulses').
diet(obesity,      'High-fibre, low-calorie food, salads and plenty of water').
diet(typhoid,      'Soft, easily digestible food such as rice, banana and boiled vegetables').

avoid(diabetes,     'Sweets, sugary drinks and refined flour').
avoid(hypertension, 'Pickles, processed food and excess salt').
avoid(anaemia,      'Tea or coffee immediately after meals').
avoid(obesity,      'Fried food, sweets and fast food').
avoid(typhoid,      'Spicy, oily and raw food').

suggest(D) :-
    diet(D, Diet), !,
    avoid(D, Avoid),
    format('Disease          : ~w~n', [D]),
    format('Recommended diet : ~w~n', [Diet]),
    format('Foods to avoid   : ~w~n', [Avoid]).
suggest(D) :-
    format('No diet information available for ~w.~n', [D]).
```

4. INPUT

```
?- suggest(diabetes).
?- suggest(hypertension).
?- suggest(typhoid).
?- suggest(fever).
```

5. OUTPUT

```
?- suggest(diabetes).
Disease          : diabetes
Recommended diet : Low-sugar foods, whole grains, green vegetables and fibre
Foods to avoid   : Sweets, sugary drinks and refined flour
true.

?- suggest(hypertension).
Disease          : hypertension
Recommended diet : Low-salt food, fruits, vegetables and low-fat dairy
Foods to avoid   : Pickles, processed food and excess salt
true.

?- suggest(typhoid).
Disease          : typhoid
Recommended diet : Soft, easily digestible food such as rice, banana and boiled vegetables
Foods to avoid   : Spicy, oily and raw food
true.
```

```
?- suggest(fever).
No diet information available for fever.
true.
```

6. RESULT

The Prolog program suggested the recommended diet and the foods to avoid for the given disease. (For educational use only; consult a dietician for real cases.)

EXPERIMENT 25

Monkey Banana Problem

1. AIM

To write a Prolog program to solve the Monkey and Banana problem: the monkey must find a sequence of actions to reach a banana hanging from the ceiling.

2. PROCEDURE

1. Represent a state as state(MonkeyPosition, OnFloorOrBox, BoxPosition, HasBanana).
2. Define the moves: grasp the banana (monkey on the box in the middle), climb the box, push the box and walk.
3. Write plan/3, a depth-first search that avoids repeating states, until the monkey has the banana.
4. Open SWI-Prolog, save the program as a .pl file and load it with ?- [filename]. (consult).
5. Query go. to print the plan from the initial state (monkey at the door, box at the window).

3. CODE (Prolog)

```
pos(atwindow).
pos(middle).
pos(atdoor).

% move(State1, Action, State2)
move(state(middle, onbox, middle, hasnot), grasp, state(middle, onbox, middle, has)).
move(state(P, onfloor, P, H), climb, state(P, onbox, P, H)).
move(state(P1, onfloor, P1, H), push(P1, P2), state(P2, onfloor, P2, H)) :-
    pos(P2), P1 \= P2.
move(state(P1, onfloor, B, H), walk(P1, P2), state(P2, onfloor, B, H)) :-
    pos(P2), P1 \= P2.

plan(state(_, _, _, has), _, []).
plan(S1, Visited, [Action|Actions]) :-
    move(S1, Action, S2),
    \+ member(S2, Visited),
    plan(S2, [S2|Visited], Actions).

write_plan([], _).
write_plan([A|As], N) :-
    format('Step ~w: ~w~n', [N, A]),
    N1 is N + 1,
    write_plan(As, N1).

go :-
    Start = state(atdoor, onfloor, atwindow, hasnot),
    plan(Start, [Start], Plan), !,
    write('Plan to get the banana:'), nl,
    write_plan(Plan, 1).
```

4. INPUT

```
?- go.
```

5. OUTPUT

```
?- go.
Plan to get the banana:
Step 1: walk(atdoor,atwindow)
Step 2: push(atwindow,middle)
Step 3: climb
Step 4: grasp
true.
```

6. RESULT

The monkey obtained the banana by walking to the box, pushing it to the middle, climbing on it and grasping the banana.

EXPERIMENT 26

Fruit and its Colour using Backtracking

1. AIM

To write a Prolog program for fruits and their colours and to demonstrate backtracking in finding all the answers.

2. PROCEDURE

1. Create the facts fruit(Name) and color(Fruit, Colour).
2. Write the rule fruit_color(F, C) combining the two predicates.
3. Write show_all/0 which prints a solution and then forces backtracking using fail to print the next solution.
4. Open SWI-Prolog, save the program as a .pl file and load it with ?- [filename]. (consult).
5. Query show_all. and findall/3 queries and observe the answers.

3. CODE (Prolog)

```
fruit(apple).
fruit(banana).
fruit(grape).
fruit(orange).
fruit(mango).
fruit(cherry).
fruit(lemon).

color(apple,  red).
color(banana, yellow).
color(grape,  purple).
color(orange, orange).
color(mango,  yellow).
color(cherry, red).
color(lemon,  yellow).

fruit_color(F, C) :- fruit(F), color(F, C).

% fail forces Prolog to backtrack and find the next solution
show_all :- fruit_color(F, C), format('~w is ~w~n', [F, C]), fail.
show_all.
```

4. INPUT

```
?- show_all.
?- findall(F, fruit_color(F, yellow), L).
?- findall(F, fruit_color(F, red), L).
?- findall(C, fruit_color(grape, C), L).
```

5. OUTPUT

```
?- show_all.
apple is red
banana is yellow
grape is purple
orange is orange
mango is yellow
cherry is red
lemon is yellow
true.

?- findall(F, fruit_color(F, yellow), L).
L = [banana, mango, lemon].

?- findall(F, fruit_color(F, red), L).
```

```
L = [apple, cherry].

?- findall(C, fruit_color(grape, C), L).
L = [purple].
```

6. RESULT

Backtracking was demonstrated: Prolog found all the fruit-colour pairs one after another.

EXPERIMENT 27

Best First Search Algorithm

1. AIM

To write a Prolog program to implement the (greedy) Best First Search algorithm.

2. PROCEDURE

1. Represent the graph by edge(Node1, Node2) facts, give a heuristic h(Node, Value) for each node and declare the goal node.
2. Keep an open list of paths and always expand the path whose first node has the smallest heuristic value.
3. Add the unvisited children of the expanded node to the open list and sort the list by heuristic value.
4. Open SWI-Prolog, save the program as a .pl file and load it with ?- [filename]. (consult).
5. Query best_first(Start, Path) and observe the nodes expanded and the path found.

3. CODE (Prolog)

```
edge(s, a). edge(s, b).
edge(a, c). edge(a, d).
edge(b, e). edge(b, f).
edge(d, g). edge(e, g).

h(s, 10). h(a, 5). h(b, 7). h(c, 6).
h(d, 4).  h(e, 3). h(f, 6). h(g, 0).

goal(g).

best_first(Start, Path) :-
    bfs([[Start]], [], RevPath),
    reverse(RevPath, Path).

bfs([[N|P]|_], _, [N|P]) :- goal(N), !.
bfs([[N|P]|Rest], Visited, Sol) :-
    h(N, H),
    format('Expanding ~w (h=~w)~n', [N, H]),
    findall([M, N|P],
            ( edge(N, M), \+ member(M, Visited), \+ member(M, [N|P]) ),
            Children),
    append(Children, Rest, Open0),
    sort_by_h(Open0, Open),
    bfs(Open, [N|Visited], Sol).

sort_by_h(Paths, Sorted) :-
    findall(H-Path, ( member(Path, Paths), Path = [N|_], h(N, H) ), Pairs),
    keysort(Pairs, SortedPairs),
    findall(Path, member(_-Path, SortedPairs), Sorted).
```

4. INPUT

```
?- best_first(s, Path).
?- best_first(b, Path).
```

5. OUTPUT

```
?- best_first(s, Path).
Expanding s (h=10)
Expanding a (h=5)
Expanding d (h=4)
Path = [s, a, d, g].

?- best_first(b, Path).
Expanding b (h=7)
Expanding e (h=3)
Path = [b, e, g].
```

6. RESULT

The Best First Search algorithm found a path from the start node to the goal node by always expanding the most promising node.

EXPERIMENT 28

Medical Diagnosis

1. AIM

To write a Prolog program for a simple rule-based medical diagnosis system that identifies a disease from the symptoms of a patient.

2. PROCEDURE

1. Store the symptoms of each patient as facts symptom(Patient, Symptom).
2. Write one rule per disease: the disease is diagnosed if all its characteristic symptoms are present.
3. Add treatment(Disease, Advice) facts for each disease.
4. Open SWI-Prolog, save the program as a .pl file and load it with ?- [filename]. (consult).
5. Query diagnose(Patient) and observe the diagnosis and the suggested treatment.

3. CODE (Prolog)

```
symptom(john, fever).     symptom(john, cough).     symptom(john, body_ache).
symptom(mary, headache).  symptom(mary, nausea).    symptom(mary, sensitivity_to_light).
symptom(ravi, fever).     symptom(ravi, rash).      symptom(ravi, joint_pain).
symptom(sita, sneezing).  symptom(sita, runny_nose).

disease(P, flu) :-
    symptom(P, fever), symptom(P, cough), symptom(P, body_ache).
disease(P, migraine) :-
    symptom(P, headache), symptom(P, nausea), symptom(P, sensitivity_to_light).
disease(P, dengue) :-
    symptom(P, fever), symptom(P, rash), symptom(P, joint_pain).
disease(P, common_cold) :-
    symptom(P, sneezing), symptom(P, runny_nose).

treatment(flu,         'Rest, plenty of fluids and paracetamol').
treatment(migraine,    'Rest in a dark, quiet room and consult a neurologist').
treatment(dengue,      'Plenty of fluids and immediate medical attention').
treatment(common_cold, 'Rest, warm fluids and steam inhalation').

diagnose(P) :-
    disease(P, D), treatment(D, T), !,
    format('~w is suffering from ~w.~n', [P, D]),
    format('Suggested treatment: ~w~n', [T]).
diagnose(P) :-
    format('No diagnosis found for ~w. Please consult a doctor.~n', [P]).
```

4. INPUT

```
?- diagnose(john).
?- diagnose(mary).
?- diagnose(ravi).
?- diagnose(sita).
?- diagnose(arun).
```

5. OUTPUT

```
?- diagnose(john).
john is suffering from flu.
Suggested treatment: Rest, plenty of fluids and paracetamol
true.

?- diagnose(mary).
mary is suffering from migraine.
Suggested treatment: Rest in a dark, quiet room and consult a neurologist
true.
```

```
?- diagnose(ravi).
ravi is suffering from dengue.
Suggested treatment: Plenty of fluids and immediate medical attention
true.

?- diagnose(sita).
sita is suffering from common_cold.
Suggested treatment: Rest, warm fluids and steam inhalation
true.

?- diagnose(arun).
No diagnosis found for arun. Please consult a doctor.
true.
```

6. RESULT

The rule-based system diagnosed the disease of each patient from the symptoms. (Educational example only, not a substitute for a doctor.)

EXPERIMENT 29

Forward Chaining

1. AIM

To write a Prolog program to implement forward chaining (data-driven reasoning) and to incorporate the required queries.

2. PROCEDURE

1. Store the known facts as dynamic facts fact(F) and the rules as rule(Conclusion, [Conditions]).
2. Find a rule whose conclusion is not yet known and whose conditions are all known facts.
3. Add the conclusion to the fact base using assertz/1 and print it.
4. Repeat until no rule can derive a new fact.
5. Open SWI-Prolog, save the program as a .pl file and load it with ?- [filename]. (consult). Query forward. and then query the fact base.

3. CODE (Prolog)

```
% Forward chaining
:- dynamic fact/1.

fact(has_fever).
fact(has_cough).
fact(has_sore_throat).

rule(flu,                [has_fever, has_cough]).
rule(needs_rest,         [flu]).
rule(see_doctor,         [flu, has_sore_throat]).
rule(prescribe_medicine, [see_doctor]).

all_facts([]).
all_facts([H|T]) :- fact(H), all_facts(T).

forward :-
    rule(C, Conds),
    \+ fact(C),
    all_facts(Conds),
    assertz(fact(C)),
    format('Derived new fact: ~w~n', [C]),
    !,
    forward.
forward.
```

4. INPUT

```
?- forward.
?- fact(prescribe_medicine).
?- findall(F, fact(F), L).
```

5. OUTPUT

```
?- forward.
Derived new fact: flu
Derived new fact: needs_rest
Derived new fact: see_doctor
Derived new fact: prescribe_medicine
true.

?- fact(prescribe_medicine).
true.

?- findall(F, fact(F), L).
L = [has_fever, has_cough, has_sore_throat, flu, needs_rest, see_doctor,
```

```
    prescribe_medicine].
```

6. RESULT

Forward chaining derived all the new facts from the initial facts and the rules.

EXPERIMENT 30

Backward Chaining

1. AIM

To write a Prolog program to implement backward chaining (goal-driven reasoning) and to incorporate the required queries.

2. PROCEDURE

1. Store the facts as fact(F) and the rules as rule(Conclusion, [Conditions]).
2. To prove a goal: it is true if it is a known fact.
3. Otherwise find a rule whose conclusion is the goal and recursively prove all its conditions (sub-goals).
4. Print the reasoning steps, and use ask/1 to report whether the goal is TRUE or FALSE.
5. Open SWI-Prolog, save the program as a .pl file and load it with ?- [filename]. (consult). Query ask(Goal).

3. CODE (Prolog)

```
% Backward chaining
fact(has_fever).
fact(has_cough).
fact(has_sore_throat).

rule(flu,                [has_fever, has_cough]).
rule(needs_rest,         [flu]).
rule(see_doctor,         [flu, has_sore_throat]).
rule(prescribe_medicine, [see_doctor]).

prove(G) :-
    fact(G),
    format('  ~w is a known fact~n', [G]).
prove(G) :-
    rule(G, Conds),
    format('Trying to prove ~w using ~w~n', [G, Conds]),
    prove_all(Conds),
    format('  ~w is proved~n', [G]).

prove_all([]).
prove_all([H|T]) :- prove(H), prove_all(T).

ask(G) :-
    (   prove(G)
    ->  format('Result: ~w is TRUE~n', [G])
    ;   format('Result: ~w is FALSE~n', [G])
    ).
```

4. INPUT

```
?- ask(prescribe_medicine).
?- ask(has_fever).
?- ask(malaria).
```

5. OUTPUT

```
?- ask(prescribe_medicine).
Trying to prove prescribe_medicine using [see_doctor]
Trying to prove see_doctor using [flu,has_sore_throat]
Trying to prove flu using [has_fever,has_cough]
  has_fever is a known fact
  has_cough is a known fact
  flu is proved
  has_sore_throat is a known fact
  see_doctor is proved
```

```
  prescribe_medicine is proved
Result: prescribe_medicine is TRUE
true.

?- ask(has_fever).
  has_fever is a known fact
Result: has_fever is TRUE
true.

?- ask(malaria).
Result: malaria is FALSE
true.
```

6. RESULT

Backward chaining proved the goal by reducing it to sub-goals that are known facts; an unprovable goal was reported FALSE.

EXPERIMENT 31

Web Blog using WordPress (Anchor Tag, Title Tag, etc.)

1. AIM

To create a web blog using WordPress and to demonstrate the use of the Anchor tag, Title tag and other HTML tags.

2. PROCEDURE

1. Install WordPress locally (XAMPP / LocalWP) or create a free blog at wordpress.com, and log in to the Dashboard.
2. Go to Settings > General and set the Site Title and Tagline of the blog (the HTML <title> tag of the site).
3. Go to Posts > Add New, enter the post title (it becomes the <title> and <h1> of the page) and add a Custom HTML block.
4. Paste the HTML code given below, which uses heading, paragraph, anchor (<a>), title attribute and list tags.
5. Click Publish and then View Post to see the blog; hover over the links to see the title tooltips and click the anchors to test them.

3. CODE (HTML / WordPress)

```
<!-- Content pasted in the "Custom HTML" block of the post -->
<!-- Post title entered in WordPress: "Artificial Intelligence: An Introduction" -->
<!-- WordPress generates:  <title>Artificial Intelligence: An Introduction - AI Lab Blog</title> -->

<h1>Artificial Intelligence: An Introduction</h1>
<p>Artificial Intelligence (AI) is the branch of computer science that builds
machines capable of intelligent behaviour such as searching, reasoning and learning.</p>

<h2>Contents</h2>
<ul>
  <li><a href="#history" title="Jump to the History section">History of AI</a></li>
  <li><a href="#tools" title="Jump to the Tools section">Tools used in the AI lab</a></li>
</ul>

<h2 id="history">History of AI</h2>
<p>The term <b>Artificial Intelligence</b> was coined in <i>1956</i> at the
Dartmouth Conference.</p>

<h2 id="tools">Tools used in the AI lab</h2>
<ul>
  <li><a href="https://www.python.org" title="Visit the Python website"
         target="_blank">Python</a></li>
  <li><a href="https://www.swi-prolog.org" title="Visit the SWI-Prolog website"
         target="_blank">SWI-Prolog</a></li>
</ul>

<p><a href="#top" title="Go back to the top of the page">Back to top</a></p>
```

4. INPUT

```
Site title: AI Lab Blog   |   Post title: Artificial Intelligence: An Introduction   |   HTML content as given in the CODE section.
```

5. OUTPUT

```
Browser tab (Title tag)  : Artificial Intelligence: An Introduction - AI Lab Blog

Page displayed by the blog:
  Artificial Intelligence: An Introduction            (Heading 1)
  Artificial Intelligence (AI) is the branch of computer science that builds
  machines capable of intelligent behaviour such as searching, reasoning
  and learning.

  Contents                                            (Heading 2)
   - History of AI              (anchor link -> scrolls to the History section)
   - Tools used in the AI lab   (anchor link -> scrolls to the Tools section)

  History of AI                                       (Heading 2, id="history")
  The term Artificial Intelligence was coined in 1956 at the Dartmouth
  Conference.      (bold and italic text)

  Tools used in the AI lab                            (Heading 2, id="tools")
   - Python       (opens https://www.python.org in a new tab)
   - SWI-Prolog   (opens https://www.swi-prolog.org in a new tab)

  Back to top     (anchor link -> returns to the top of the page)

Hovering the mouse over any link shows its title attribute as a tooltip,
e.g. "Visit the Python website".
```

6. RESULT

A WordPress blog post was created and published, demonstrating the title tag, anchor tags (internal and external links), the title attribute and other HTML tags.

EXPERIMENT 32

Pattern Matching

1. AIM

To write a Prolog program to implement pattern matching on lists and atoms (finding a pattern inside a list or a word).

2. PROCEDURE

1. Represent the text and the pattern as lists (atoms are converted to lists of characters with atom_chars/2).
2. A pattern occurs in a list if the list can be split using append/3 into Before + Pattern + After.
3. Because Prolog unifies, a pattern may contain variables, which are bound to the matching elements.
4. Open SWI-Prolog, save the program as a .pl file and load it with ?- [filename]. (consult).
5. Query match_list/2, match_atom/2 and position/3 and observe the answers.

3. CODE (Prolog)

```
% match_list(Pattern, List): Pattern occurs as a sub-list of List
match_list(Pattern, List) :-
    append(_, Rest, List),
    append(Pattern, _, Rest), !.

% match_atom(Pattern, Text): pattern atom occurs inside the text atom
match_atom(Pattern, Text) :-
    atom_chars(Pattern, P),
    atom_chars(Text, T),
    match_list(P, T).

% position(Pattern, Text, Pos): Pos is the (0-based) position of each occurrence
position(Pattern, Text, Pos) :-
    atom_chars(Pattern, P),
    atom_chars(Text, T),
    append(Before, Rest, T),
    append(P, _, Rest),
    length(Before, Pos).
```

4. INPUT

```
?- match_list([a, X, c], [x, a, b, c, y]).
?- match_atom(ell, hello).
?- match_atom(xyz, hello).
?- findall(P, position(ana, banana, P), L).
?- findall(P, position(a, banana, P), L).
```

5. OUTPUT

```
?- match_list([a, X, c], [x, a, b, c, y]).
X = b.

?- match_atom(ell, hello).
true.

?- match_atom(xyz, hello).
false.

?- findall(P, position(ana, banana, P), L).
L = [1, 3].

?- findall(P, position(a, banana, P), L).
L = [1, 3, 5].
```

6. RESULT

Pattern matching was implemented in Prolog; patterns (with variables) were located in lists and words.

EXPERIMENT 33

Number of Vowels

1. AIM

To write a Prolog program to find the number of vowels in a given word.

2. PROCEDURE

1. Define the facts vowel(a), vowel(e), vowel(i), vowel(o), vowel(u).
2. Convert the word to lower case and then to a list of characters.
3. Recursively traverse the list: if the head is a vowel, add 1 to the count of the tail; otherwise just count the tail.
4. Open SWI-Prolog, save the program as a .pl file and load it with ?- [filename]. (consult).
5. Query count_vowels_in_word(Word, N) with different words and observe N.

3. CODE (Prolog)

```
vowel(a).
vowel(e).
vowel(i).
vowel(o).
vowel(u).

count_vowels([], 0).
count_vowels([H|T], N) :-
    vowel(H), !,
    count_vowels(T, N1),
    N is N1 + 1.
count_vowels([_|T], N) :-
    count_vowels(T, N).

count_vowels_in_word(Word, N) :-
    downcase_atom(Word, Lower),
    atom_chars(Lower, Chars),
    count_vowels(Chars, N).
```

4. INPUT

```
?- count_vowels_in_word(artificial, N).
?- count_vowels_in_word('Intelligence', N).
?- count_vowels_in_word(prolog, N).
?- count_vowels_in_word(rhythm, N).
```

5. OUTPUT

```
?- count_vowels_in_word(artificial, N).
N = 5.

?- count_vowels_in_word('Intelligence', N).
N = 5.

?- count_vowels_in_word(prolog, N).
N = 2.

?- count_vowels_in_word(rhythm, N).
N = 0.
```

6. RESULT

The number of vowels in the given words was counted successfully using Prolog.